

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5

5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I B Aishwarya/192421432(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. STUDENT COURSE(SID, SName, CID, CName, Instructor, Grade)

Functional Dependencies

Assume:

- Each student has one name.
- Each course has one course name and instructor.
- A student's grade is determined by the student and course.

Therefore:

1. **SID $\rightarrow$ SName**
2. **CID $\rightarrow$ CName**
3. **CID $\rightarrow$ Instructor**
4. **SID, CID $\rightarrow$ Grade**

So,

(SID, CID) $\rightarrow$ SName, CName, Instructor, Grade

Candidate Key

Find the closure of {SID, CID}:

(SID, CID)⁺ = {SID, CID, SName, CName, Instructor, Grade}

Therefore, **{SID, CID}** determines all attributes.

Answer

Candidate Key = (SID, CID)

The relation contains partial dependencies:

- SID $\rightarrow$ SName
- CID $\rightarrow$ CName, Instructor

Hence, it is not in 2NF if the key is composite.

```
MySQL 8.0 Command Line Cli
Enter password: ****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 47
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.03 sec)

mysql> USE college_db;
Database changed
mysql>
mysql> CREATE TABLE STUDENT (
->     StudentID INT,
->     StudentName VARCHAR(50),
->     Email VARCHAR(100),
->     Age INT
->     DepartmentID INT
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'D
epartmentID INT
)' at line 6
mysql> CREATE TABLE STUDENT (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50) NOT NULL,
->     Email VARCHAR(100) UNIQUE,
->     Age INT CHECK (Age >= 17),
->     DepartmentID INT
-> );
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE STUDENT (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50) NOT NULL,
```

```
mysql> CREATE TABLE STUDENT (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50) NOT NULL,
->     Email VARCHAR(100) UNIQUE,
->     Age INT CHECK (Age >= 17),
->     DepartmentID INT
-> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> DESC STUDENT;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| StudentID | int | NO | PRI | NULL | |
| StudentName | varchar(50) | NO | | NULL | |
| Email | varchar(100) | YES | UNI | NULL | |
| Age | int | YES | | NULL | |
| DepartmentID | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> |
```

Q2. Convert an Unnormalized Relation into 1NF

Consider the following unnormalized relation:

UNF

SID	SName	Courses
101	Aishwarya	C101-DBMS-A, C102-OS-B
102	Priya	C101-DBMS-A

The **Courses** attribute contains multiple values, so it violates 1NF.

Step 1: Identify repeating groups

For student 101:

Courses = {C101-DBMS-A, C102-OS-B}

This is a multivalued/repeating attribute.

Step 2: Convert to atomic values

Create one row for each course.

1NF Relation

SID	SName	CID	CName	Grade
101	Aishwarya	C101	DBMS	A
101	Aishwarya	C102	OS	B
102	Priya	C101	DBMS	A

Justification

A relation is in 1NF when:

- Every attribute contains a single atomic value.
- There are no repeating groups or multivalued attributes.
- Each row represents one record.

Therefore, the above relation is in 1NF.

```
mysql> CREATE TABLE STUDENT (  
-> StudentID INT PRIMARY KEY,  
-> StudentName VARCHAR(50) NOT NULL,  
-> Email VARCHAR(100) UNIQUE,  
-> Age INT CHECK (Age >= 17),  
-> DepartmentID INT  
-> );  
ERROR 1050 (42S01): Table 'student' already exists  
mysql> DESC STUDENT;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| StudentID | int | NO | PRI | NULL | |  
| StudentName | varchar(50) | NO | | NULL | |  
| Email | varchar(100) | YES | UNI | NULL | |  
| Age | int | YES | | NULL | |  
| DepartmentID | int | YES | | NULL | |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.01 sec)  
  
mysql> |
```

Q3. R(A,B,C,D,E)

Given FDs:

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Step 1: Find the candidate key

Calculate A^+ :

$$A^+ = \{A\}$$

Using $A \rightarrow BC$:

$$A^+ = \{A, B, C\}$$

Using $BC \rightarrow D$:

$$A^+ = \{A, B, C, D\}$$

Using $D \rightarrow E$:

$$A^+ = \{A, B, C, D, E\}$$

Therefore:

$$A^+ = ABCDE$$

So, **A is the candidate key.**

Step 2: Check 2NF

2NF removes partial dependencies on part of a composite key.

Here, the candidate key is A, which is a single attribute.

Therefore, a partial dependency is impossible.

Conclusion

R is already in 2NF.

There is no decomposition required for 2NF.

However, there are transitive dependencies:

$$A \rightarrow BC \rightarrow D \rightarrow E$$

So the relation is not necessarily in 3NF.

Final Answer

Candidate Key: A

2NF: Yes

Partial dependency: None

2NF decomposition: Not required.

```
mysql> SELECT Student_ID, StudentNmae, DepartmntID
      -> FROM STUDENT;
ERROR 1054 (42S22): Unknown column 'Student_ID' in 'field list'
mysql> SELECT StudentID, StudentName, DepartmentID
      -> FROM STUDENT;
```

StudentID	StudentName	DepartmentID
1	Arun	101
2	Priya	102
3	Ravi	101
4	Divya	103
5	Kiran	102

5 rows in set (0.00 sec)

Q4. 3NF Decomposition

Relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Assume the FDs are:

1. **EmpID → EmpName, DeptID**
2. **DeptID → DeptName, ManagerID**

Step 1: Identify candidate key

EmpID+

= {EmpID}

Using EmpID → EmpName, DeptID:

= {EmpID, EmpName, DeptID}

Using DeptID → DeptName, ManagerID:

= {EmpID, EmpName, DeptID, DeptName, ManagerID}

Therefore:

Candidate Key = EmpID

Step 2: Identify transitive dependency

We have:

EmpID → DeptID

and

DeptID → DeptName, ManagerID

Therefore:

EmpID → DeptName, ManagerID is a transitive dependency.

Step 3: Decompose

Create:

EMPLOYEE

EmpID EmpName DeptID

Primary Key: EmpID

DEPARTMENT

DeptID DeptName ManagerID

Primary Key: DeptID

Foreign Key: DeptID in EMPLOYEE references DEPARTMENT.

Result

The transitive dependency is removed.

Therefore, the decomposition is in 3NF.

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> CREATE TABLE DEPARTMENT (
->   DepartmentID INT PRIMARY KEY,
->   DepartmentName VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO DEPARTMENT
-> VALUES
-> (101, 'Computer Science'),
-> (102, 'Electronics'),
-> (103, 'Mechanical');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT StudentName, DepartmentName
-> FROM STUDENT
-> JOIN DEPARTMENT;
+-----+-----+
| StudentName | DepartmentName |
+-----+-----+
| Arun        | Mechanical     |
| Arun        | Electronics    |
| Arun        | Computer Science |
| Priya       | Mechanical     |
| Priya       | Electronics    |
| Priya       | Computer Science |
| Ravi        | Mechanical     |
| Ravi        | Electronics    |
| Ravi        | Computer Science |
| Divya       | Mechanical     |
| Divya       | Electronics    |
| Divya       | Computer Science |
| Kiran       | Mechanical     |
| Kiran       | Electronics    |
| Kiran       | Computer Science |
+-----+-----+
15 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> CREATE TABLE DEPARTMENT (
->   DepartmentID INT PRIMARY KEY,
->   DepartmentName VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO DEPARTMENT
-> VALUES
-> (101, 'Computer Science'),
-> (102, 'Electronics'),
-> (103, 'Mechanical');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT StudentName, DepartmentName
-> FROM STUDENT
-> JOIN DEPARTMENT;
+-----+-----+
| StudentName | DepartmentName |
+-----+-----+
| Arun        | Mechanical     |
| Arun        | Electronics    |
| Arun        | Computer Science |
| Priya       | Mechanical     |
| Priya       | Electronics    |
| Priya       | Computer Science |
| Ravi        | Mechanical     |
| Ravi        | Electronics    |
| Ravi        | Computer Science |
| Divya       | Mechanical     |
| Divya       | Electronics    |
| Divya       | Computer Science |
| Kiran       | Mechanical     |
| Kiran       | Electronics    |
| Kiran       | Computer Science |
+-----+-----+
15 rows in set (0.00 sec)
```

Q5. BCNF Decomposition

Relation:

R(A,B,C,D)

FDs:

- $AB \rightarrow C$
- $C \rightarrow D$
- $D \rightarrow A$

Step 1: Find candidate keys

AB⁺

$AB \rightarrow C$

Then $C \rightarrow D$

Therefore:

$AB^+ = \{A,B,C,D\}$

So **AB is a candidate key.**

BC⁺

$B,C \rightarrow D$ using $C \rightarrow D$.

$D \rightarrow A$.

Therefore:

$BC^+ = \{A,B,C,D\}$

So **BC is a candidate key.**

BD⁺

$D \rightarrow A$

Therefore:

$BD^+ = \{A,B,D\}$

Cannot derive C.

So BD is not a key.

Step 2: Check BCNF

For BCNF, every non-trivial FD $X \rightarrow Y$ must have **X as a superkey.**

Consider:

$C \rightarrow D$

C is **not a superkey**.

Therefore, the relation **violates BCNF**.

Also:

$D \rightarrow A$

D is not a superkey.

Therefore, BCNF is violated.

Step 3: Decompose using $C \rightarrow D$

Create:

$R_1(C,D)$

and

$R_2(A,B,C)$

R_1

C D

C is the key.

R_2

A B C

AB is a key.

Final Answer

$R \rightarrow R_1(C,D) + R_2(A,B,C)$

Both relations satisfy BCNF.

```

mysql> SELECT StudentName
-> FROM STUDENT
-> WHERE DepartmentID = (
->     SELECT DepartmentID
->     FROM DEPARTMENT
-> );
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> SELECT StudentName
-> FROM STUDENT
-> WHERE DepartmentID IN (
->     SELECT DepartmentID
->     FROM DEPARTMENT
-> );
+-----+
| StudentName |
+-----+
| Arun        |
| Priya       |
| Ravi        |
| Divya       |
| Kiran       |
+-----+
5 rows in set (0.01 sec)

```

Q6. Lossless-Join Decomposition Using Chase Algorithm

Consider:

R(A,B,C)

FD:

$A \rightarrow B$

Decompose into:

- R1(A,B)
- R2(A,C)

Chase Table

Row	A	B	C
R1	a	b	c ₁
R2	a	b ₂	c

Since both rows have the same value **A = a** and the FD is:

$A \rightarrow B$

we make the B values equal:

Row	A	B	C
R1	a	b	c ₁
R2	a	b	c

The rows can be joined to produce the original relation.

Conclusion

Since the chase produces a row containing the original attribute values, the decomposition is **lossless-join**.

Therefore:

$R(A,B,C) \rightarrow R1(A,B)$ and $R2(A,C)$ is lossless.

```
mysql> SELECT StudentName
-> FROM STUDENT
-> WHERE DepartmentID = (
->     SELECT DepartmentID
->     FROM DEPARTMENT
-> );
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> SELECT StudentName
-> FROM STUDENT
-> WHERE DepartmentID IN (
->     SELECT DepartmentID
->     FROM DEPARTMENT
-> );
+-----+
| StudentName |
+-----+
| Arun        |
| Priya       |
| Ravi        |
| Divya       |
| Kiran       |
+-----+
5 rows in set (0.01 sec)
```

Q7. MVDs and 4NF

Relation:

TEACHER(TID, Subject, Hobby)

Assume a teacher can teach several subjects and have several hobbies independently.

Therefore:

TID $\rightarrow\rightarrow$ Subject

and

TID $\rightarrow\rightarrow$ Hobby

These are **multivalued dependencies (MVDs)**.

Example

TID	Subject	Hobby
T1	DBMS	Music
T1	DBMS	Cricket
T1	OS	Music
T1	OS	Cricket

This causes redundancy because every subject is combined with every hobby.

4NF Decomposition

Decompose into:

TEACHER_SUBJECT

TID Subject

TEACHER_HOBBY

TID Hobby

Result

TEACHER(TID, Subject, Hobby)

↓

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

The independent multivalued facts are separated.

Therefore, the decomposed relations are in 4NF.

```
mysql> SELECT *
-> FROM STUDENT
-> WHERE Age >= 18
-> AND Age > 18
-> AND DepartmentID = 101
-> AND DepartmentID = 101;
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Email | Age | DepartmentID |
+-----+-----+-----+-----+-----+
| 1 | Arun | arun@gmail.com | 20 | 101 |
| 3 | Ravi | ravi@gmail.com | 21 | 101 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT *
-> FROM STUDENT
-> WHERE Age > 18
-> AND DepartmentID = 101;
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Email | Age | DepartmentID |
+-----+-----+-----+-----+-----+
| 1 | Arun | arun@gmail.com | 20 | 101 |
| 3 | Ravi | ravi@gmail.com | 21 | 101 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q8. Join Dependency and 5NF

Consider:

SPJ(Supplier, Part, Project)

Suppose a supplier supplies a part to a project only when these three independent relationships exist:

- Supplier–Part
- Supplier–Project
- Part–Project

The original table can contain redundant combinations.

Example

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S1	P1	J2

If the relationships are independent, storing all combinations creates redundancy.

Join Dependency

The relation has a join dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

Therefore, decompose into:

SP

SP(Supplier, Part)

SJ

SJ(Supplier, Project)

PJ

PJ(Part, Project)

5NF Result

The original relation can be reconstructed by:

$SP \bowtie SJ \bowtie PJ$

Thus, decomposition removes redundancy caused by the join dependency.

Conclusion

5NF ensures that every non-trivial join dependency is implied by candidate keys.

```
mysql> CREATE TABLE STUDENT (  
-> StudentID INT PRIMARY KEY,  
-> StudentName VARCHAR(50) NOT NULL,  
-> Email VARCHAR(100) UNIQUE,  
-> Age INT CHECK (Age >= 17),  
-> DepartmentID INT  
-> );  
ERROR 1050 (42S01): Table 'student' already exists  
mysql> INSERT INTO STUDENT  
-> VALUES (6, NULL, 'new@gmail.com', 16, 101);  
ERROR 1048 (23000): Column 'StudentName' cannot be null  
mysql> INSERT INTO STUDENT  
-> VALUES (6, 'Sanjay', 'sanjay@gmail.com', 18, 101);  
Query OK, 1 row affected (0.01 sec)  
  
mysql> INSERT INTO STUDENT  
-> VALUES (7, 'Ravi', 'ravi2@gmail.com', 20, 101);  
Query OK, 1 row affected (0.01 sec)
```

Q9. College Database – Anomalies and 3NF

Consider a poorly designed table:

COLLEGE(StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName, Grade)

Example:

StudentID	StudentName	CourseID	CourseName	FacultyID	FacultyName	Grade
101	Aishwarya	C1	DBMS	F1	Kumar	A
101	Aishwarya	C2	OS	F2	Ravi	B
102	Priya	C1	DBMS	F1	Kumar	A

Functional Dependencies

1. StudentID $\rightarrow$ StudentName
2. CourseID $\rightarrow$ CourseName, FacultyID
3. FacultyID $\rightarrow$ FacultyName
4. StudentID, CourseID $\rightarrow$ Grade

Anomalies

1. Update anomaly

If faculty Kumar's name changes, several records must be updated.

2. Insertion anomaly

A new course cannot be inserted easily unless a student is also registered.

3. Deletion anomaly

Deleting the last student enrolled in a course may also delete information about that course and faculty.

3NF Decomposition

Create:

STUDENT

STUDENT(StudentID, StudentName)

COURSE

COURSE(CourseID, CourseName, FacultyID)

FACULTY

FACULTY(FacultyID, FacultyName)

ENROLLMENT

ENROLLMENT(StudentID, CourseID, Grade)

Keys

- STUDENT → StudentID
- COURSE → CourseID
- FACULTY → FacultyID
- ENROLLMENT → (StudentID, CourseID)

Result

The decomposition removes:

- Update anomaly
- Insertion anomaly
- Deletion anomaly
- Data redundancy

Therefore, the database is normalized to 3NF.

```
MySQL 8.0 Command Line CLI  x  +  v

mysql> STUDENT(SID, SName, PhoneNumbers)
-> ;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'STUDENT(SID, SName, PhoneNumbers)' at line 1
mysql> CREATE TABLE STUDENT_1NF (
->   SID INT PRIMARY KEY,
->   SName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE STUDENT_PHONE (
->   SID INT,
->   PhoneNumber VARCHAR(15),
->   PRIMARY KEY (SID, PhoneNumber),
->   FOREIGN KEY (SID) REFERENCES STUDENT_1NF(SID)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO STUDENT_1NF
-> VALUES
-> (1, 'Arun'),
-> (2, 'Priya');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO STUDENT_PHONE
-> VALUES
-> (1, '9876543210'),
-> (1, '9123456780'),
-> (2, '9988776655');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE STUDENT_2NF (
->   StudentID INT PRIMARY KEY,
->   StudentName VARCHAR(50)
-> );
```

```

mysql> INSERT INTO STUDENT_PHONE
-> VALUES
-> (1, '9876543210'),
-> (1, '9123456780'),
-> (2, '9988776655');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE STUDENT_2NF (
-> StudentID INT PRIMARY KEY,
-> StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE COURSE (
-> CourseID INT PRIMARY KEY,
-> CourseName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE ENROLLMENT (
-> StudentID INT,
-> CourseID INT,
-> Marks INT,
-> PRIMARY KEY (StudentID, CourseID),
-> FOREIGN KEY (StudentID) REFERENCES STUDENT_2NF(StudentID),
-> FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
-> );
Query OK, 0 rows affected (0.09 sec)

```

Q10. Attribute Closure and Candidate Keys

For a relation $R(A,B,C,D,E)$, suppose the FDs are:

- $A \rightarrow B$
- $B \rightarrow C$
- $CD \rightarrow E$
- $E \rightarrow A$

Step 1: Calculate A^+

Start:

$A^+ = \{A\}$

$A \rightarrow B$:

$\{A,B\}$

$B \rightarrow C$:

$\{A,B,C\}$

Cannot derive D.

Therefore:

$A^+ = ABC$

A is not a key.

Step 2: Calculate AD^+

Start:

$$AD^+ = \{A, D\}$$

$A \rightarrow B$:

$$\{A, B, D\}$$

$B \rightarrow C$:

$$\{A, B, C, D\}$$

$CD \rightarrow E$:

$$\{A, B, C, D, E\}$$

Therefore:

$$AD^+ = ABCDE$$

So **AD** is a candidate key.

Step 3: Calculate CD^+

Start:

$$CD^+ = \{C, D\}$$

$CD \rightarrow E$:

$$\{C, D, E\}$$

$E \rightarrow A$:

$$\{A, C, D, E\}$$

$A \rightarrow B$:

$$\{A, B, C, D, E\}$$

Therefore:

$$CD^+ = ABCDE$$

So **CD** is a candidate key.

Step 4: Calculate DE^+

Start:

$$DE^+ = \{D, E\}$$

$E \rightarrow A$:

{A,D,E}

$A \rightarrow B$:

{A,B,D,E}

$B \rightarrow C$:

{A,B,C,D,E}

Therefore:

DE is a candidate key.

Final Answer

Candidate Keys:

- **AD**
- **CD**
- **DE**

The attribute closure method is used to verify whether an attribute set determines all attributes of the relation.

```
mysql> INSERT INTO STUDENT_PHONE
-> VALUES
-> (1, '9876543210'),
-> (1, '9123456780'),
-> (2, '9988776655');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE STUDENT_2NF (
-> StudentID INT PRIMARY KEY,
-> StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE COURSE (
-> CourseID INT PRIMARY KEY,
-> CourseName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE ENROLLMENT (
-> StudentID INT,
-> CourseID INT,
-> Marks INT,
-> PRIMARY KEY (StudentID, CourseID),
-> FOREIGN KEY (StudentID) REFERENCES STUDENT_2NF(StudentID),
-> FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
-> );
Query OK, 0 rows affected (0.09 sec)
```

```

MySQL 8.0 Command Line Cli  x  +  v

mysql> CREATE TABLE STUDENT_3NF (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50),
->     DepartmentID INT,
->     FOREIGN KEY (DepartmentID)
->         REFERENCES DEPARTMENT_3NF(DepartmentID)
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO STUDENT_3NF
-> VALUES
-> (1, 'Arun', 101),
-> (2, 'Priya', 102),
-> (3, 'Ravi', 101);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_3NF;
+-----+-----+-----+
| StudentID | StudentName | DepartmentID |
+-----+-----+-----+
| 1 | Arun | 101 |
| 2 | Priya | 102 |
| 3 | Ravi | 101 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT
->     S.StudentID,
->     S.StudentName,
->     D.DepartmentName,
->     D.DepartmentLocation
-> FROM STUDENT_3NF S
-> JOIN DEPARTMENT_3NF D
-> ON S.DepartmentID = D.DepartmentID;
+-----+-----+-----+-----+
| StudentID | StudentName | DepartmentName | DepartmentLocation |
+-----+-----+-----+-----+
| 1 | Arun | Computer Science | Block A |
| 2 | Priya | Electronics | Block B |
+-----+-----+-----+-----+

```

```

mysql> SELECT
->     S.StudentID,
->     S.StudentName,
->     D.DepartmentName,
->     D.DepartmentLocation
-> FROM STUDENT_3NF S
-> JOIN DEPARTMENT_3NF D
-> ON S.DepartmentID = D.DepartmentID;
+-----+-----+-----+-----+
| StudentID | StudentName | DepartmentName | DepartmentLocation |
+-----+-----+-----+-----+
| 1 | Arun | Computer Science | Block A |
| 2 | Priya | Electronics | Block B |
| 3 | Ravi | Computer Science | Block A |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```